

Modeling Risk- Quantitative Analysis

Montecarlo Simulation For Quantifying Risk

```
In [1]: import random
import pandas as pd
import altair as alt
import numpy as np
```

```
In [5]: # Handle large data
alt.data_transformers.enable('default', max_rows=None)
```

```
Out[5]: DataTransformerRegistry.enable('default')
```

Monte Carlo Simulation for Cost Profiling

Here, we can run simulations to model:

- Production cost variations
- Demand fluctuation
- Market entry risks

With this, we will be able to trace the distribution of possible outcomes, allowing better budgeting, contingency planning, and risk communication as compared to a single metricS reliance (For example, a decision on a single number like average cost is risky).

Case 1

Note: Consider a company which is launching a new product with expected cost structure highlited as below. The company faces significant uncertainty in estimating the total cost due to variable factors such as production, logistics, and market readiness. Relying on single-point estimates risks underbudgeting or overbudgeting. Therefore, the challenge is to quantify and manage the cost risk using a probabilistic approach that can inform more confident and resilient budgeting decisions. Perform a montecarlo sumulation that will help quantify the risk profile of the company.This is a roughly developed sketch and feel free to play around the analysis using an actual data.

```
In [6]: #Given conditions for the cost structure of the company:

optimistic = random.randint(0, 5000)
most_likely = random.randint(5000, 10000)
pessimistic = random.randint(10000, 100000)
```

```
print("optimistic = ",optimistic, "most_likely=",most_likely, "pessimistic =",
total_cost = (optimistic + 4*most_likely + pessimistic)/6 # (beta) Program Evalu
```

```
optimistic = 2464 most_likely = 6261 pessimistic = 65605
```

In [7]: *# Monte Carlo simulation with 10,000 iterations*

```
iteration = 10000

def montecarlo(iteration):
    optimistic = np.random.randint(0, 5000, iteration)
    most_likely = np.random.randint(5000, 10000, iteration)
    pessimistic = np.random.randint(10000, 20000, iteration)
    cost = (optimistic + 4*most_likely + pessimistic) / 6
    return cost

cost = montecarlo(iteration)
cost_df = pd.DataFrame({'Cost': cost})
cost_df
```

Out[7]:

	Cost
0	7386.166667
1	8717.166667
2	7963.833333
3	6514.500000
4	9483.166667
...	...
9995	6491.500000
9996	7515.333333
9997	7997.666667
9998	5863.500000
9999	8935.666667

10000 rows × 1 columns

In [8]: *#Summary stat to discribe the data and variability in costs*

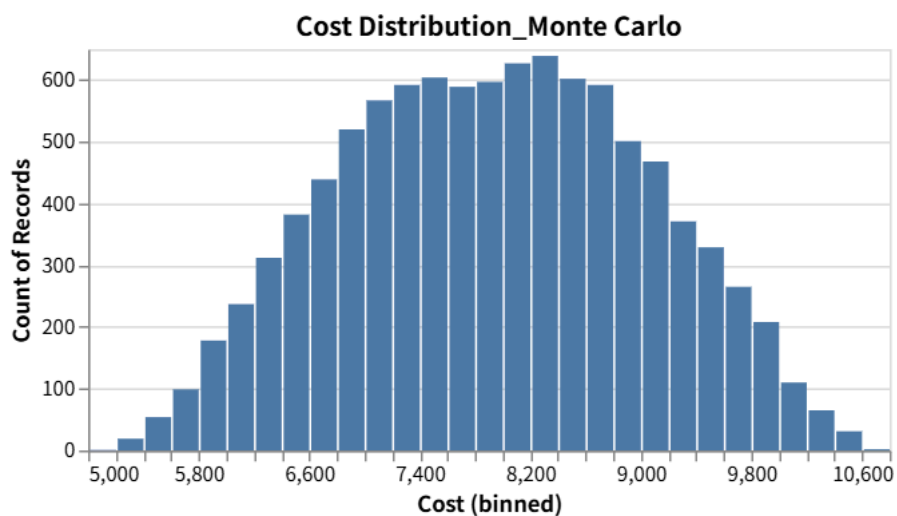
```
cost_df.describe()
```

Out[8]:

	Cost
count	10000.000000
mean	7927.034000
std	1099.344948
min	5186.166667
25%	7092.333333
50%	7938.833333
75%	8748.791667
max	10687.333333

```
In [9]: #Distribution plot of risk - with a normaly distributed risk, the tails of the d
alt.Chart(cost_df).mark_bar().encode(
    alt.X('Cost', bin=alt.Bin(maxbins=50)),
    y='count()'
).properties(
    title='Cost Distribution_Monte Carlo',
    width= 400,
    height= 200
)
```

Out[9]:



Discussion

Distribution plot of risk - with a normaly distributed risk, the tails of the distribution depict outliers to consider possibilities for rare events to consider (1) Rare low-cost scenarios such as smooth execution, unexpected vendor discounts, or avoided production issues: can lead to underbudgeting and overconfidence if relied upon. (2) Rare high-cost scenarios such as costs exceeding \$1000 due to supply chain disruptions, regulatory delays, unforeseen technical issues, or sudden inflation: pose significant budget overrun risks and must be planned for using contingency funds or insurance.

```
In [10]: # Cumulative probability
```

```

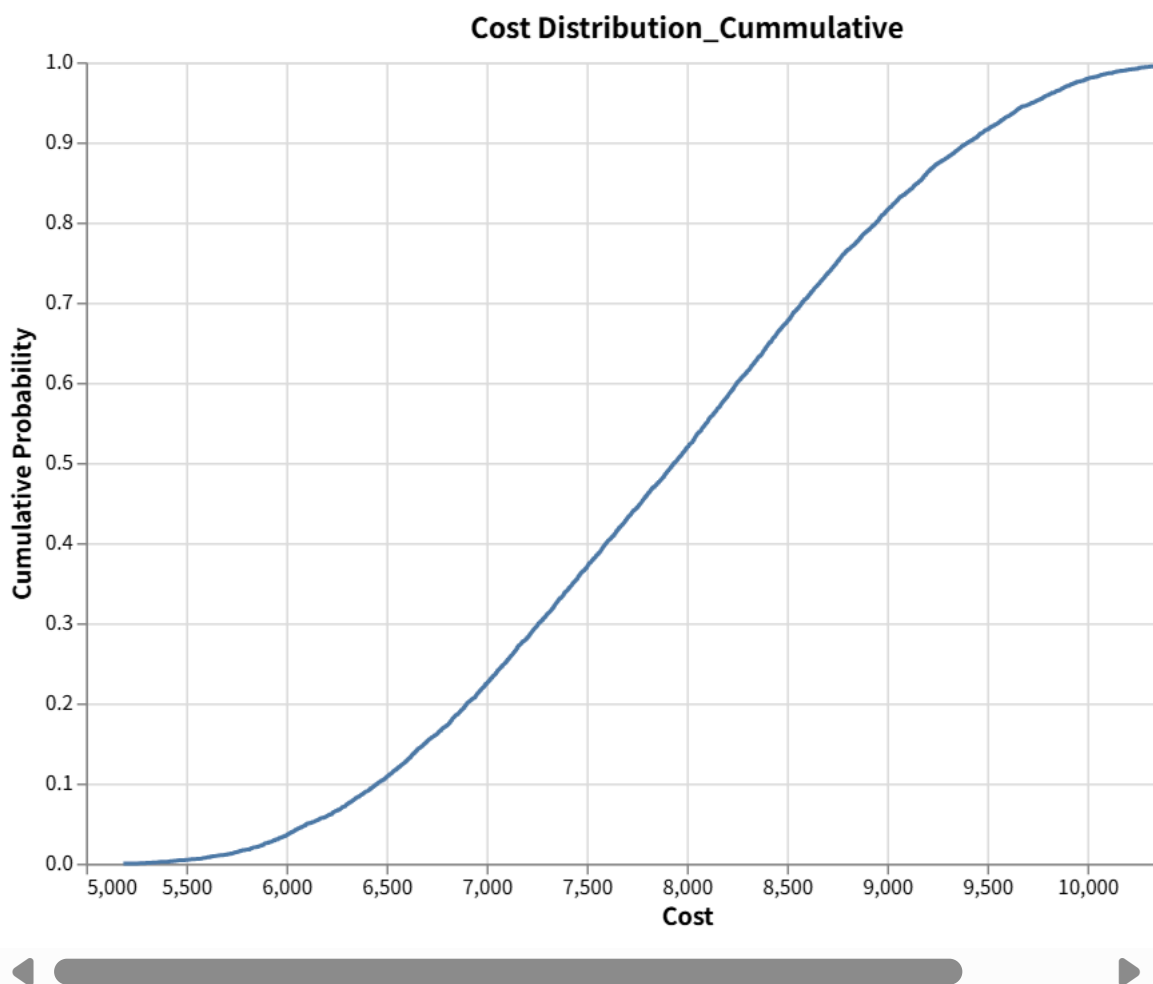
# Here, commulative distribution curve might be handy indefining the risk tresho
#avoide based on the proportion of cummulative risks: Say for example that you a
#risk of upto 90%, what level of cost should you incur to make this investment
cost_df = cost_df.sort_values(by='Cost').reset_index(drop=True)
cost_df['cumulative_cost'] = (np.arange(len(cost_df)) + 1) / len(cost_df)

chart = alt.Chart(cost_df).mark_line().encode(
    x=alt.X('Cost', title='Cost'),
    y=alt.Y('cumulative_cost', title='Cumulative Probability')
).properties(
    title='Cost Distribution_Cummulative',
    width=600,
    height=400
).configure_axis(
    titleFontSize=12,
    labelFontSize=10
).configure_title(
    fontSize=14
)

chart

```

Out[10]:



Discussions

The result depicts: the probabilistic cost profile of launching a new product based on a Monte Carlo simulation using PERT assumptions (optimistic, most likely, and pessimistic estimates). For example, What's the maximum cost we can expect with

90% confidence? Based on the simulation, \$9500 covers 90% of cost scenarios (i.e. a risk-adjusted budget recommendation balancing ambition with caution to support a financially viable and resilient product launch strategy). If we want to be that confident, we should set the budget at 950k.

Takeaways

Risk Thresholds: Use the CDF to define decision boundaries (e.g., 90th percentile cost).

Risk Appetite: Align cost thresholds with stakeholder risk tolerance.

Monte Carlo Strength: Shows not just the average, but the full spread of outcomes.

Tail Planning: Important for setting reserve funds or insurance.

Case 2

Case 2: *GIGO*

!! Monte Carlo simulation is only as good as the assumptions you feed.

It is very important to properly quantify the possible scenarios for the optimistic, most_likely and pessimistic outcomes out of the investment decision. Consider the following risk profile of the company that significantly changes the risk profile of the company. See below, for example, for optimistic, most likely and pessimistic costs in 1000s:

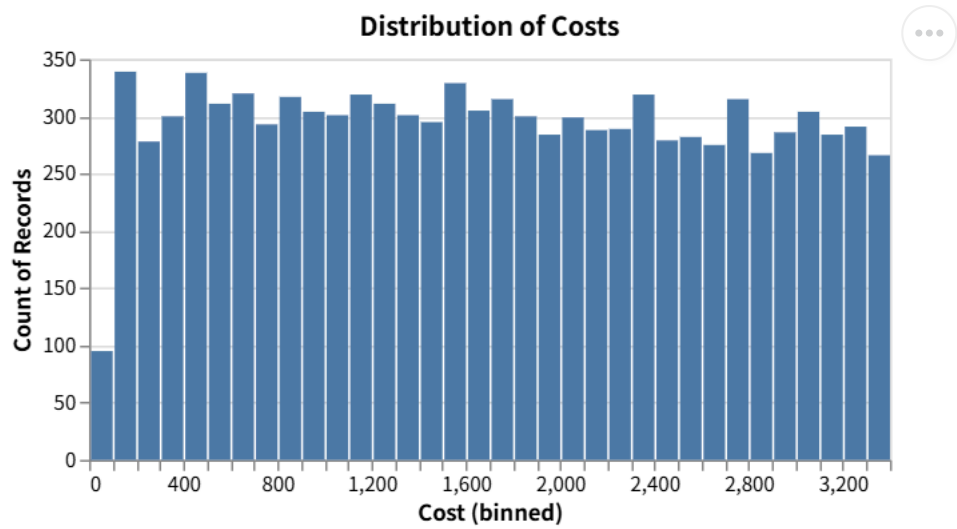
```
In [11]: iteration = 10000

def montecarlo(iteration):
    optimistic = np.random.randint(0, 50, iteration)
    most_likely = np.random.randint(50, 100, iteration)
    pessimistic = np.random.randint(100, 20000, iteration)
    cost = (optimistic + 4*most_likely + pessimistic) / 6
    return cost

cost = montecarlo(iteration)
cost_df = pd.DataFrame({'Cost': cost})

alt.Chart(cost_df).mark_bar().encode(
    alt.X('Cost', bin=alt.Bin(maxbins=50)),
    y='count()'
).properties(
    title='Distribution of Costs',
    width= 400,
    height= 200
)
```

Out[11]:



In []:

In []: